

Taller — Primeros pasos con Scikit-learn, TensorFlow y PyTorch

Versión PROFESOR (soluciones y guía de facilitación)

Notas generales para el profesor:

- Tiempo sugerido: 60-90 min (10 min Paso 0, 20 min Parte 1, 20 min Parte 2, 20 min Parte 3, 10-20 min Parte 4 + cierre).
- Todo el código de este taller fue probado en un entorno con `scikit-learn 1.9`, `tensorflow 2.21` (Keras 3) y `torch 2.13`. Si el curso usa versiones distintas, revisa especialmente el aviso del Ejercicio 2.1 (ver más abajo).
- La idea pedagógica central: los tres *frameworks* resuelven el **mismo problema** ($y = 2x$) en la Parte 1/2/3, para que el contraste de líneas de código, control y abstracción sea directamente observable — no solo teórico.
- Este taller es la continuación práctica de la actividad del anexo ("probar el *framework* en 3 minutos") y de la pregunta orientadora del foro de la Semana 3.

Paso 0 — Preparación

```
import sklearn
import tensorflow as tf
import torch

print("scikit-learn:", sklearn.__version__)
print("tensorflow:", tf.__version__)
print("torch:", torch.__version__)
```

Respuesta esperada — Pregunta de reflexión 0: `pip install scikit-learn`, `pip install tensorflow`, `pip install torch` (o `pip install -r requirements.txt` si el curso distribuye un archivo de dependencias). Aprovecha para recordar que en Colab los tres ya vienen preinstalados.

Error común: estudiantes con Python muy reciente o muy antiguo pueden tener conflictos de versión con `tensorflow`. Si ocurre, sugiere usar Google Colab para evitar perder tiempo de clase en instalación.

Parte 1 — Scikit-learn: machine learning clásico

Ejercicio 1.1 — Solución

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

X = [[1], [2], [3], [4]]
```

```
y = [2, 4, 6, 8]

modelo = LinearRegression()
modelo.fit(X, y)

print("prediccion:", modelo.predict([[5]]))
print("mse:", mean_squared_error(y, modelo.predict(X)))
```

Salida esperada: `prediccion: [10.]`, `mse: 0.0` (el problema es perfectamente lineal, así que el error de entrenamiento es cero).

Ejercicio 1.2 — Solución

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

clf = KNeighborsClassifier(n_neighbors=3)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

print("accuracy:", accuracy_score(y_test, y_pred))
```

Salida esperada: `accuracy` cercano a 1.0 (iris es un dataset pequeño y muy separable; con `random_state=42` y `n_neighbors=3` suele dar 100%).

Respuesta esperada — Pregunta de reflexión 1: Con scikit-learn, pasar de datos a modelo entrenado toma muy pocas líneas gracias a la API uniforme `fit/predict`, disponible para prácticamente cualquier algoritmo clásico. Sobre interpretabilidad: con `KNeighborsClassifier` la explicación es directa ("la flor se clasificó como *setosa* porque sus 3 vecinos más cercanos en el conjunto de entrenamiento eran de esa especie"), a diferencia de una red neuronal donde es más difícil trazar una explicación tan concreta.

Punto de discusión en clase: preguntar qué pasaría con la interpretabilidad si en vez de `KNeighborsClassifier` usaran un modelo de árbol (`DecisionTreeClassifier`) — conecta con el criterio técnico "interpretabilidad" visto en las diapositivas.

Parte 2 — TensorFlow/Keras: deep learning de alto nivel

Ejercicio 2.1 — Solución

```
import numpy as np
import tensorflow as tf

X = np.array([1.0, 2.0, 3.0, 4.0])
y = np.array([2.0, 4.0, 6.0, 8.0])

modelo = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(1),
])
modelo.compile(optimizer="sgd", loss="mse")
modelo.fit(X, y, epochs=200, verbose=0)

print(modelo.predict(np.array([5.0])))
```

Salida esperada: un valor cercano a `[[10.]]` (no exacto, porque a diferencia de scikit-learn, Keras ajusta los pesos por descenso de gradiente iterativo, no por una solución analítica exacta).

Aviso importante de compatibilidad: con Keras 3 (TensorFlow ≥ 2.16), pasar los datos como listas de Python planas (`X = [1.0, 2.0, 3.0, 4.0]`) produce el error `ValueError: Unrecognized data type`. Es un buen momento para señalar en clase que **no todos los frameworks aceptan los datos de la misma forma** — de nuevo, el criterio técnico "preprocesamiento" de las diapositivas. Si algún estudiante copia el ejemplo de una versión anterior de Keras (con listas), este es el error que verá y así se explica.

Ejercicio 2.2 — Solución

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X = digits.data / 16.0
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

clf = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(64,)),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax"),
])
clf.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=
["accuracy"])
clf.fit(X_train, y_train, epochs=20, verbose=0)

loss, acc = clf.evaluate(X_test, y_test, verbose=0)
print("accuracy digits:", acc)
```

Salida esperada: **accuracy** típicamente entre 0.93 y 0.98 (varía ligeramente por la inicialización aleatoria de los pesos).

Respuesta esperada — Pregunta de reflexión 2: El Ejercicio 1.2 (scikit-learn) resuelve la clasificación en ~4 líneas funcionales; el Ejercicio 2.2 (Keras) necesita definir explícitamente la arquitectura de la red (capas, neuronas, funciones de activación) y los hiperparámetros de entrenamiento (optimizador, épocas). Esto refleja que scikit-learn abstrae **el algoritmo completo** (uno elige el modelo, no lo diseña), mientras que Keras da control sobre **la arquitectura de la red**, porque en deep learning esa arquitectura es parte central del problema a resolver.

Sugerencia de facilitación: preguntar cuánto tardó en ejecutarse cada entrenamiento (Ejercicio 1.2 vs. 2.2) — normalmente Keras tarda notoriamente más incluso en CPU, lo que introduce de forma natural el criterio "soporte GPU/rendimiento".

Parte 3 — PyTorch: control explícito del entrenamiento

Ejercicio 3.1 — Solución

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.ones(3)
print("suma:", a + b)
print("forma:", a.shape)
print("cuda disponible:", torch.cuda.is_available())
```

Salida esperada: `suma: tensor([2., 3., 4.])`, `forma: torch.Size([3])`. `cuda disponible` será `False` en la mayoría de portátiles sin GPU dedicada — es un resultado válido y esperado, no un error.

Ejercicio 3.2 — Solución

```
import torch
import torch.nn as nn

X = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([2.0, 4.0, 6.0, 8.0])

modelo = nn.Linear(1, 1)
opt = torch.optim.SGD(modelo.parameters(), lr=0.01)
perdida = nn.MSELoss()

for _ in range(200):
    opt.zero_grad()
    error = perdida(modelo(X), y)
    error.backward()
    opt.step()
```

```
print(modelo(torch.tensor([[5.0]])))
```

Salida esperada: un valor cercano a **10** (el resultado exacto varía levemente según la inicialización aleatoria de los pesos de `nn.Linear`).

Respuesta esperada — Pregunta de reflexión 3: Ventaja: escribir cada paso (`zero_grad`, `backward`, `step`) da control y visibilidad total sobre el proceso de entrenamiento, lo que facilita depurar, modificar o experimentar con partes específicas (por ejemplo, aplicar un paso de optimización distinto a ciertas capas). Desventaja: para tareas estándar es más código y más oportunidades de cometer errores (por ejemplo, olvidar `zero_grad()` y acumular gradientes de iteraciones anteriores, un error muy común en PyTorch).

Error común a señalar: si un estudiante olvida `opt.zero_grad()`, los gradientes se acumulan entre iteraciones y el entrenamiento diverge o se vuelve inestable — vale la pena mostrarlo en vivo comentando esa línea.

Parte 4 — Reto integrador (opcional) — Guía de cierre

No hay una única respuesta correcta para la tabla comparativa; el objetivo es que el estudiante **verbalice con sus propias palabras** lo que observó al resolver el mismo problema tres veces. Como referencia orientativa:

Criterio	Scikit-learn	TensorFlow	PyTorch
Líneas de código aproximadas	Muy pocas (~4-5)	Moderadas (~8-10)	Moderadas-altas (~10-12)
Facilidad de entrenamiento	Muy alta (una línea: <code>fit</code>)	Alta (<code>fit</code> con hiperparámetros)	Media (bucle manual)
Nivel de control sobre el entrenamiento	Bajo	Medio	Alto
¿Lo usarías en producción?	Sí, para modelos clásicos/tabulares	Sí, tiene un ecosistema maduro de despliegue	Posible, pero suele requerir pasos adicionales

Respuesta esperada — Pregunta de cierre: No hay una respuesta única correcta; se espera que el estudiante justifique su elección según el tipo de problema (datos tabulares vs. imágenes/texto), el tamaño de los datos, la necesidad de explicar el modelo a terceros (interpretabilidad) o los planes de llevarlo a producción. Lo importante es que la respuesta conecte explícitamente con al menos uno de los seis criterios técnicos trabajados en la Semana 3, y no sea una preferencia sin argumentar.

Cierre sugerido de la sesión: recalcar que no existe un *framework* "mejor" en abstracto — la elección depende del problema, los datos y el contexto de aplicación, que es exactamente el eje de la pregunta orientadora del foro.

Rúbrica rápida de participación (sugerida)

Criterio	Logrado
Verifica correctamente las versiones instaladas de los tres <i>frameworks</i>	☐
Completa el Ejercicio 1.1 y 1.2 y responde la reflexión sobre interpretabilidad	☐
Completa el Ejercicio 2.1 y 2.2, usando <code>np.array</code> (no listas planas)	☐
Completa el Ejercicio 3.1 y 3.2, sin omitir <code>zero_grad()</code>	☐
Llena la tabla comparativa y argumenta la pregunta de cierre con un criterio técnico concreto	☐